

.NET 程序设计课程

期末项目：基于 .NET 的 AI Agent 开发

项目考核要求 · Agent 原理讲解 · 开发资料汇编 · 评价标准

2025–2026 学年

一、项目考核要求

1.1 项目名称

基于 .NET 的 AI Agent 开发

学生需自选一个实际问题领域，利用 .NET 技术栈开发一个能够自主推理、规划并使用工具完成任务的 AI Agent。

1.2 必须包含的核心要素

要素	说明
LLM 集成	通过 API 调用大语言模型（OpenAI / Azure OpenAI / 本地模型等）
推理循环 (Agent Loop)	实现 Thought → Action → Observation 的迭代循环
工具调用 (Tool Calling)	Agent 至少集成 3 个自定义工具/插件
记忆机制 (Memory)	实现对话记忆或工作记忆，维持上下文
用户交互界面	控制台 / Web / 桌面 / 移动端 均可

1.3 技术要求

- 基础要求（所有学生）：
- 使用 C# / .NET 8+ 开发
 - 使用 Semantic Kernel 或直接 HTTP 调用 LLM API
 - 自定义 Plugin/Tool（通过 Semantic Kernel Plugin 或原生函数注册）
 - 异步编程（async/await）的正确使用
 - 基本的错误处理与日志记录

- 进阶要求（选做，作为加分项）：
- 多 Agent 协作（Multi-Agent）
 - RAG（检索增强生成）集成外部知识库
 - MCP（Model Context Protocol）服务器实现
 - 流式输出（Streaming）
 - Agent 行为的可观测性（追踪推理过程）

1.4 交付物

交付物	说明
源代码 + README	完整可编译的项目，含构建和运行说明
架构设计文档	包含 Agent 架构图、工具设计、推理流程图
反思报告（关键）	详细说明 Agent 内部工作原理、设计决策、AI 工具使用情况
现场演示 + 答辩	10 分钟演示 + 5 分钟提问

1.5 关于 AI 使用的规则

允许使用 AI 辅助开发，但有明确要求：

- 必须在报告中如实记录 AI 工具的使用范围（哪些代码由 AI 生成、哪些是自己编写）
- 必须能解释项目中每一行代码的作用（答辩时会抽查）
- 反思报告中必须包含：对 Agent 核心循环代码的逐行解读
- AI 生成的代码如果存在 bug 且学生无法解释，将扣分

二、AI Agent 内部工作原理（教学要点）

2.1 Agent 的核心架构

AI Agent 由四个核心组件构成：LLM（大脑）、Agent Loop（控制循环）、Memory（记忆）、Tools（工具）。其中 Agent Loop 是 Agent 区别于普通 LLM 对话的关键：它让 LLM 能够反复思考、行动、观察结果，直到任务完成。

2.2 ReAct 模式（Reasoning + Acting）

ReAct 是当前 Agent 最基本的工作模式，学生必须理解并能手动实现：

循环流程：

- 1. Thought（思考）：LLM 分析当前状态，决定下一步
- 2. Action（行动）：调用一个工具/函数
- 3. Observation（观察）：获取工具返回的结果
- 4. 重复，直到任务完成或达到最大步数

核心循环伪代码（C#）：

```
async Task<string> RunAgentLoop(string userGoal)
{
    var messages = new List<ChatMessage>
    {
        new(SystemPrompt),
        new(User, userGoal)
    };

    for (int step = 0; step < MaxSteps; step++)
    {
        // 1. 调用 LLM 获取下一步行动
        var response = await llm.ChatAsync(messages);

        // 2. 如果 LLM 认为任务完成，返回结果
        if (response.FinishReason == "stop")
            return response.Content;

        // 3. 解析 LLM 请求的工具调用
        var toolCall = ParseToolCall(response);

        // 4. 执行工具
        var observation = await ExecuteTool(toolCall);

        // 5. 将结果加入上下文，继续循环
        messages.Add(response);
        messages.Add(new ToolResult(toolCall.Id, observation));
    }
    return "达到最大步数限制";
}
```

}

2.3 关键组件详解

(1) System Prompt 工程

Agent 的“人格”和“能力边界”定义。ReAct 格式的 prompt 模板要求 LLM 以 Thought/Action 格式输出。良好的 System Prompt 是 Agent 行为可控的基础。

(2) 工具定义与注册 (Function Calling)

将 C# 方法描述为 LLM 可理解的工具 (JSON Schema)。Semantic Kernel 通过 [KernelFunction] 特性自动完成这一映射。这是连接 LLM “思考”和“行动”的桥梁。

(3) 记忆 (Memory)

- 短期记忆：对话历史 (messages 列表)
- 工作记忆：当前任务的中间状态
- 长期记忆：向量数据库存储的知识 (进阶)

(4) 规划 (Planning)

- 简单：单步 ReAct 循环
- 进阶：先分解任务为子目标，再逐个执行
- Handlebars / YAML Planner (Semantic Kernel 内置)

2.4 超越 ReAct 的架构模式 (可选教学内容)

模式	特点
Plan-and-Execute	先生成完整计划，再逐步执行，可动态修正
Multi-Agent	多个专业化 Agent 协作 (如 Planner + Coder + Reviewer)
Reflection	Agent 自我反思并改进输出质量
Workflow/State Machine	用确定性状态机编排 Agent 步骤

三、.NET Agent 开发资料

3.1 核心框架

框架	说明
Microsoft Semantic Kernel	.NET AI 开发的官方 SDK, 已 GA, 支持 Agent、Plugin、Memory
Microsoft Agent Framework	2025年统一了 AutoGen + Semantic Kernel 的新框架
MCP C# SDK	Model Context Protocol 的官方 C# SDK (与 Anthropic 合作开发)

3.2 学习路径 (建议推荐给学生)

入门阶段:

- Semantic Kernel Agent Framework 官方文档: <https://learn.microsoft.com/en-us/semantic-kernel/frameworks/agent/>
- Global AI Bootcamp 2025 - Semantic Kernel C# 视频: <https://www.youtube.com/watch?v=BeAJZAURIUo>
- Configuring Agents with Plugins: <https://learn.microsoft.com/en-us/semantic-kernel/frameworks/agent/agent-f>

实践阶段:

- Building AI Agents with Semantic Kernel: <https://nanddeepn.github.io/posts/2025-04-16-ai-agents-sk/>
- Production-Ready AI Agents - Clean Architecture: <https://dev.to/sebastiandevelops/building-production-ready-a>
- MCP + Semantic Kernel 集成: <https://pub.towardsai.net/beyond-ai-agents-in-net-building-tool-connected-ai-wi>

进阶阶段:

- Multi-Agent System 实战指南: <https://medium.com/microsoftazure/step-by-step-guide-to-develop-ai-multi-age>
- Microsoft 官方培训模块: <https://learn.microsoft.com/en-us/training/modules/develop-ai-agent-with-semantic-ke>
- MCP C# SDK - GitHub: <https://github.com/modelcontextprotocol/csharp-sdk>
- Microsoft Agent Framework 概览: <https://learn.microsoft.com/en-us/agent-framework/overview/>
- Anthropic - Building Effective AI Agents: <https://anthropic.com/research/building-effective-agents>
- IBM - What is a ReAct Agent: <https://www.ibm.com/think/topics/react-agent>

3.3 NuGet 包参考

```

<!-- 基础包 -->
<PackageReference Include="Microsoft.SemanticKernel" Version="1.*" />
<PackageReference Include="Microsoft.SemanticKernel.Agents.Core" Version="1.*" />

<!-- MCP 支持 -->
<PackageReference Include="ModelContextProtocol" Version="1.*" />

<!-- 向量存储 (记忆) -->
<PackageReference Include="Microsoft.SemanticKernel.Connectors.Qdrant" Version="1.*" />

```

四、项目评价标准（100 分制）

4.1 评分维度概览

维度	分值	评价标准
Agent 核心功能	30 分	能否正确完成目标任务？推理循环是否有效？工具调用是否准确？
技术实现质量	20 分	代码架构是否合理？异步编程是否正确？错误处理是否完善？SOLID？
.NET 技术深度	15 分	是否充分利用 .NET 特性（DI、配置系统、日志等）？
架构设计文档	15 分	架构图是否清晰？工具设计是否合理？推理流程是否有说明？
反思报告与答辩	20 分	能否清晰解释原理？能否逐行解读核心代码？能否回答原理性问题？

4.2 Agent 核心功能评分细则（30分）

等级	分数	标准
优秀	25-30	Agent 稳定完成复杂任务，多步推理正确，边界情况处理良好
良好	20-24	Agent 能完成基本任务，推理循环正常工作，偶有小问题
及格	15-19	Agent 能运行但功能有限，推理有瑕疵
不及格	<15	Agent 无法正常运行或无实质推理能力

4.3 反思报告与答辩评分细则（20分）—— 防止纯 AI 代做的关键

考察点	具体问题示例
原理解释	“请解释你的 Agent Loop 的执行流程” “ReAct 模式是什么？”
代码理解	“这段工具调用代码做了什么？” “为什么用 async/await？”
设计决策	“为什么选择这些工具？” “记忆策略是如何设计的？”
AI 使用透明度	“哪些代码是 AI 生成的？你做了什么修改？为什么？”
问题诊断	给一个有 bug 的代码片段，让学生诊断问题

4.4 加分项（最多 +10 分）

加分项	分值	说明
多 Agent 协作	+3	实现多个 Agent 分工协作完成任务
RAG / 向量检索集成	+3	集成外部知识库，实现检索增强生成
MCP 服务器实现	+2	实现 Model Context Protocol 服务端
单元测试	+2	为核心组件编写完善的单元测试
创新性应用	+2	应用场景具有创新性或实用价值

五、推荐的 Agent 项目示例方向

以下为学生参考选题，学生也可自行选择其他方向：

- 1. 代码审查 Agent — 自动审查提交的代码，给出改进建议

六、教学建议

- 提供 scaffold 代码：给学生一个最小可运行的 Agent 模板（只包含基本循环），让他们在此基础上扩展
- 设置中期检查：要求学生提交架构设计和技术选型方案，防止最后赶工
- 限制 API Key 风险：学校统一提供 Azure OpenAI 资源，或允许学生使用免费的本地模型（如 Ollama + llama)
- 防作弊关键在答辩：允许用 AI 开发，但答辩时深度提问确保理解

核心设计思路：AI 能帮你写代码，但解释不了你对代码的理解。答辩和反思报告是确保学习效果的关键防线。